



3 ОКТЯБРЯ 2026 Г.

ПРАКТИЧЕСКОЕ ЗАДАНИЕ 1 ПО
ПРЕДМЕТУ «ТЕХНОЛОГИИ
ПРОГРАММИРОВАНИЯ»

РАБОТУ ВЫПОЛНИЛ КУРСАНТ ЗД РОТЫ ГРУППЫ 2122 КУРБАТОВ
ДМИТРИЙ



Пункт 1. Инициализация репозитория

Условие

Создайте новый локальный репозиторий.

Ход работы

Создал рабочую папку для лабораторной работы и открыл её в редакторе кода. Через встроенный терминал запустил команду инициализации локального репозитория. Теперь Git отслеживает изменения в этом каталоге.

Результат:

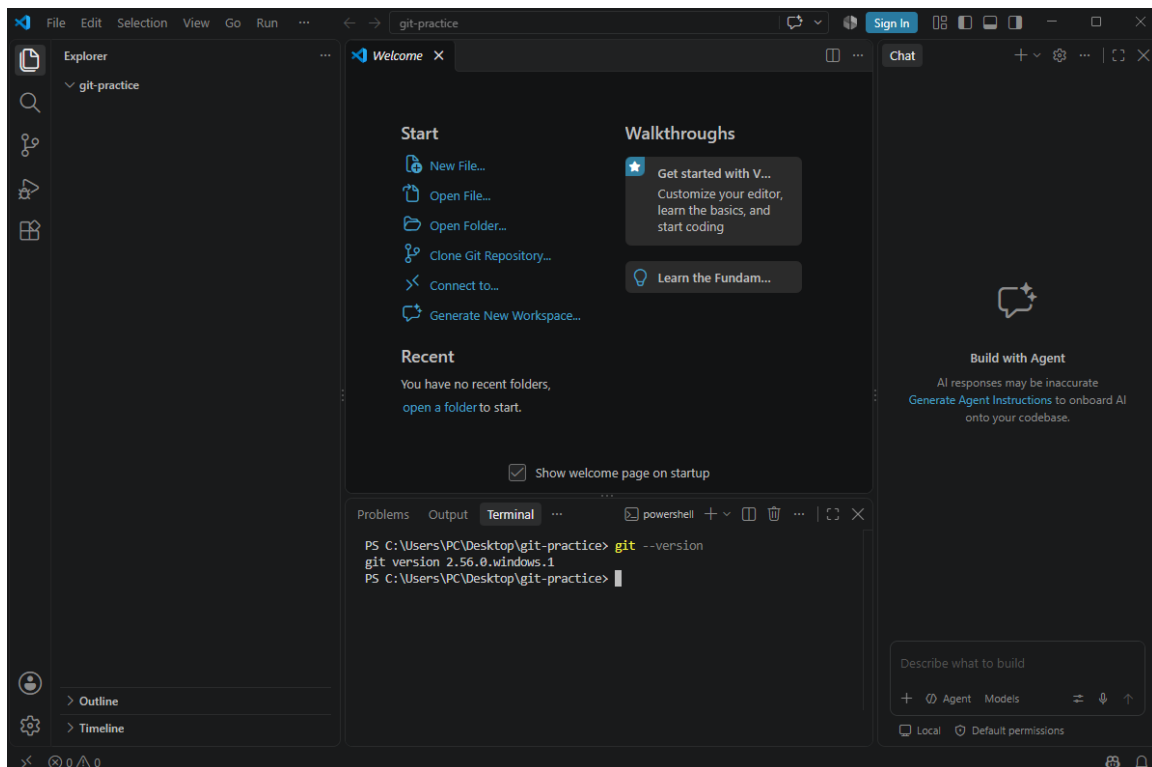


Рисунок 1. Результат пункта 1

Пункт 2. Создание файлов и первый коммит

Условие

Создайте два текстовых файла, добавьте их в репозиторий и сделайте первый коммит.

Ход работы

В корне папки создал два текстовых файла: file1.txt и file2.txt, наполнил их базовым текстом. После этого добавил оба файла в индекс и сделал первый коммит с понятным комментарием.

Результат:

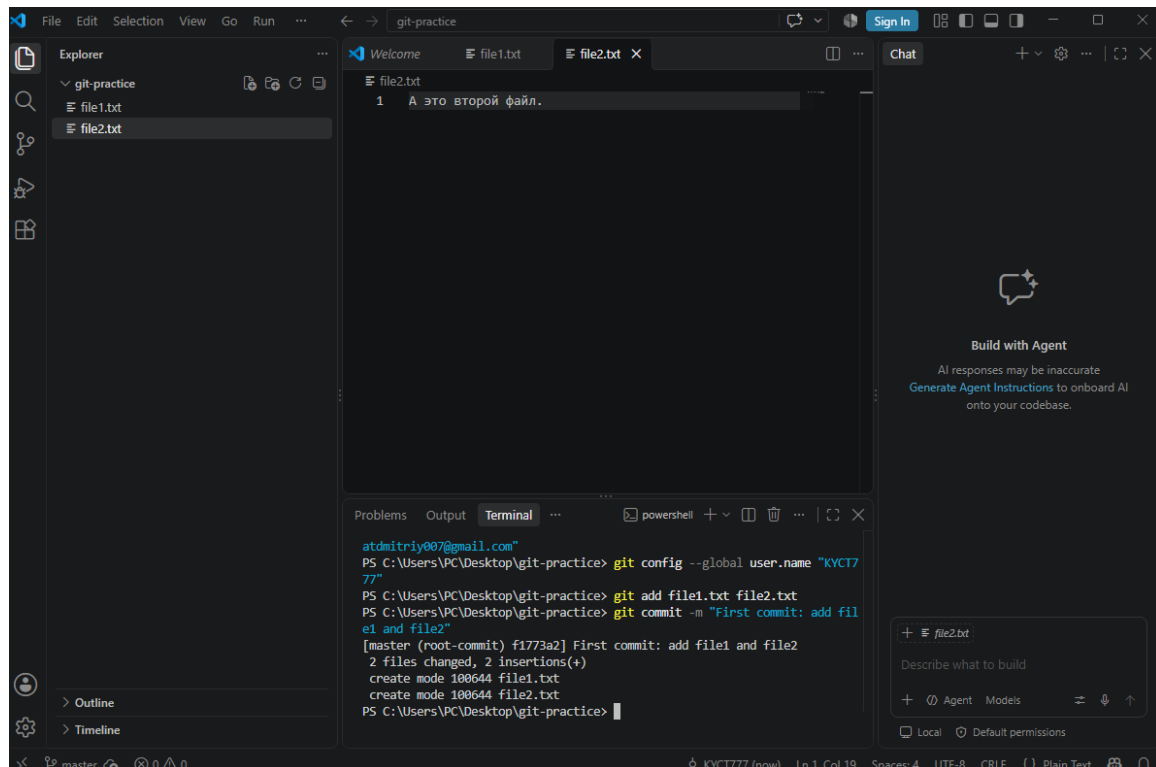


Рисунок 2. Результат пункта 2

Пункт 3. Настройка файла .gitignore

Условие

Добавьте файл gitignore, в который включите шаблон для игнорирования определённых файлов.

Ход работы

Создал служебный файл .gitignore, чтобы исключить из отслеживания лишний мусор. Прописал в него маску *.log для игнорирования файлов журналов, после чего зафиксировал изменения коммитом.

Результат:

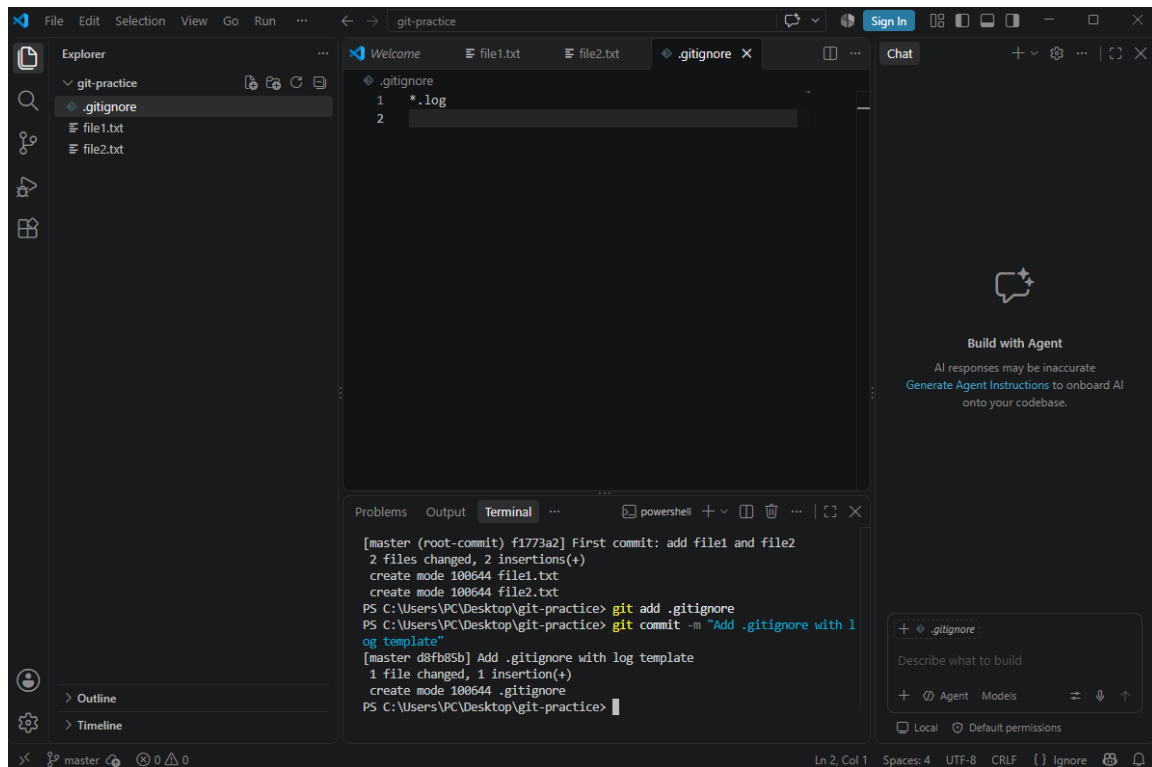


Рисунок 3. Результат пункта 3

Пункт 4. Изменение файла и второй коммит

Условие

Измените один из отслеживаемых файлов и сделайте второй коммит.

Ход работы

Открыл сохраненный ранее file1.txt, дописал туда новую строку текста и сохранил файл. Затем проиндексировал изменения и сделал второй коммит в ветку master.

Результат:

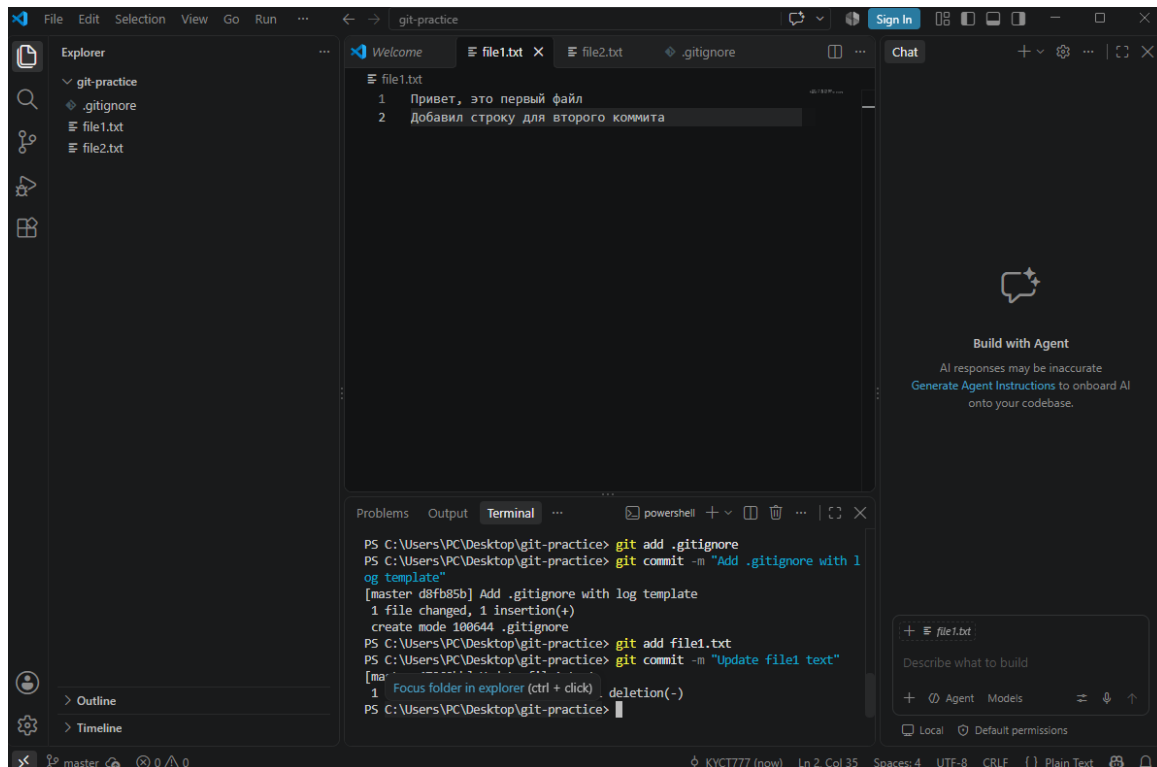


Рисунок 4. Результат пункта 4

Пункт 5. Создание и переход в новую ветку

Условие

Создайте новую ветку для изменений. Переключитесь в неё, внесите изменения в другой файл и сделайте коммит

Ход работы

Создал отдельную ветку разработки под названием development и сразу переключился на неё. Открыл файл file2.txt, внес туда правки от имени новой ветки и зафиксировал их коммитом.

Результат:

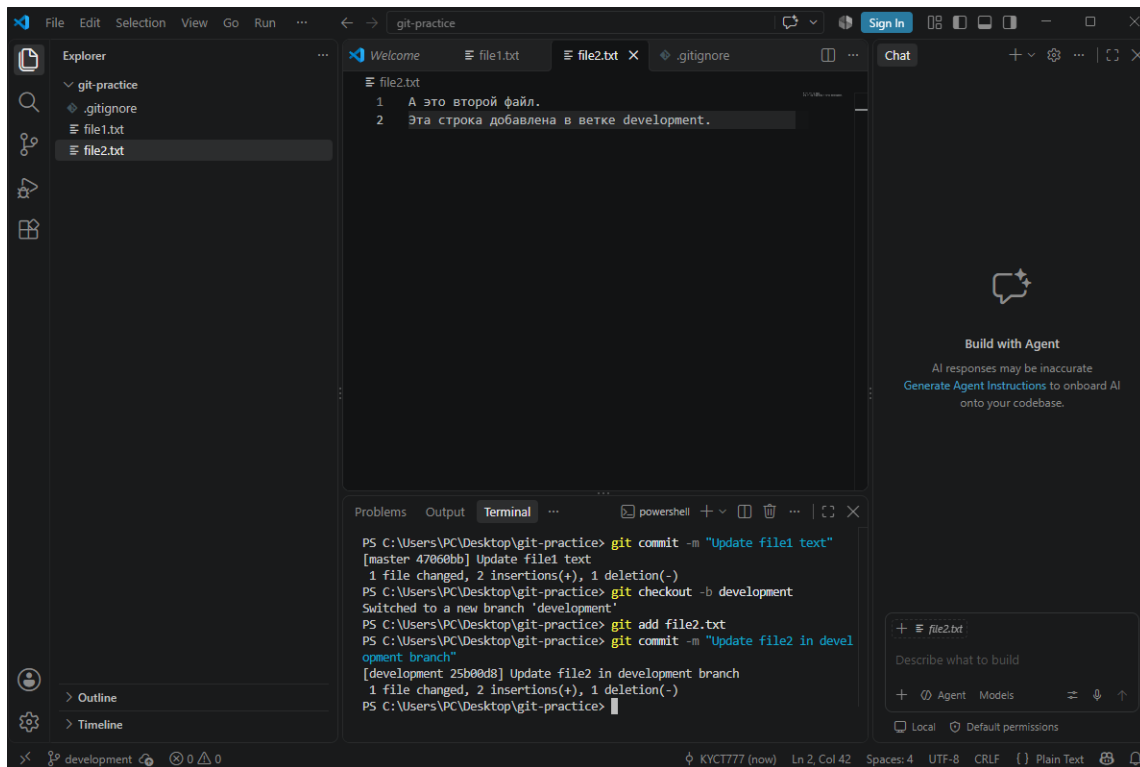


Рисунок 5. Результат пункта 5

Пункт 6. Возврат в master и слияние Fast-Forward

Условие

Вернитесь в основную ветку и выполните слияние с новой веткой. Слияние должно быть fast-forward (без конфликтов).

Ход работы

Перешел обратно в основную ветку проекта. Запустил слияние с веткой development. Так как ветка master не менялась, Git выполнил чистое слияние по типу Fast-Forward (быстрая перемотка). Правки перенеслись автоматически

Результат:

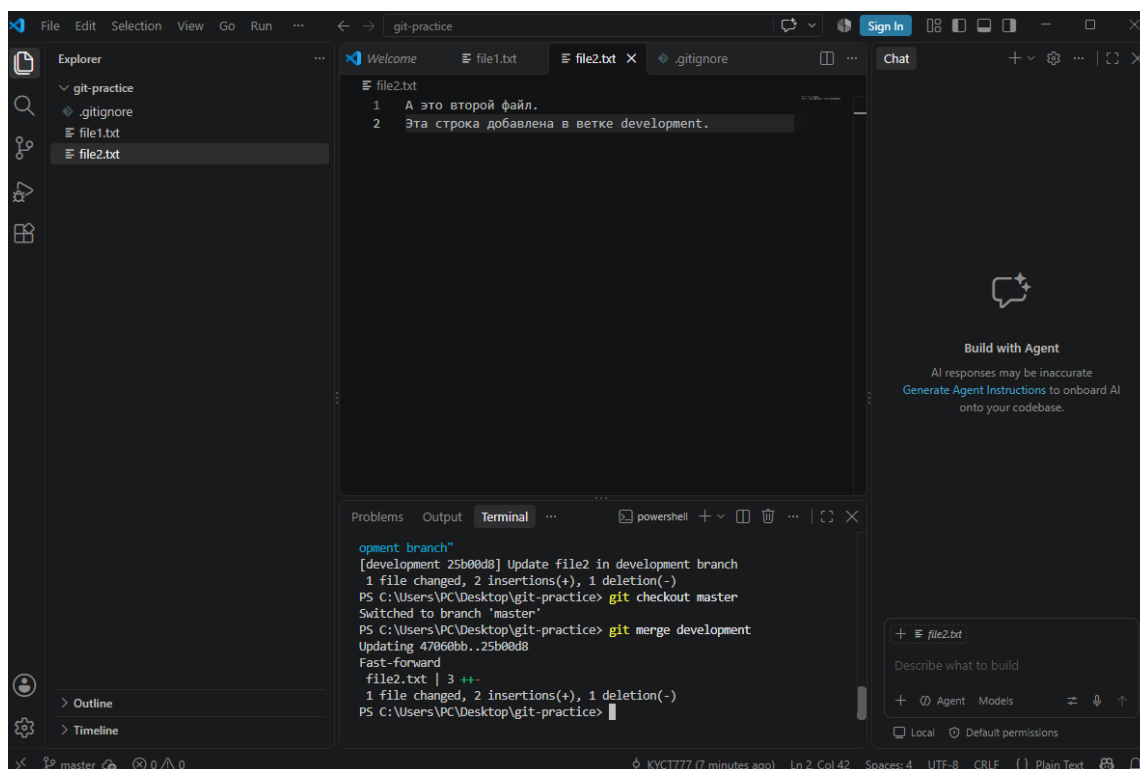


Рисунок 6. Результат пункта 6

Пункт 7. Изменение в основной ветке

Условие

Сделайте ещё одно изменение в основной ветке и зафиксируйте его коммитом.

Ход работы

Для имитации будущей совместной работы открыл file1.txt в ветке master и добавил новую строку на третью позицию. Сохранил файл и сделал коммит изменений.

Результат:

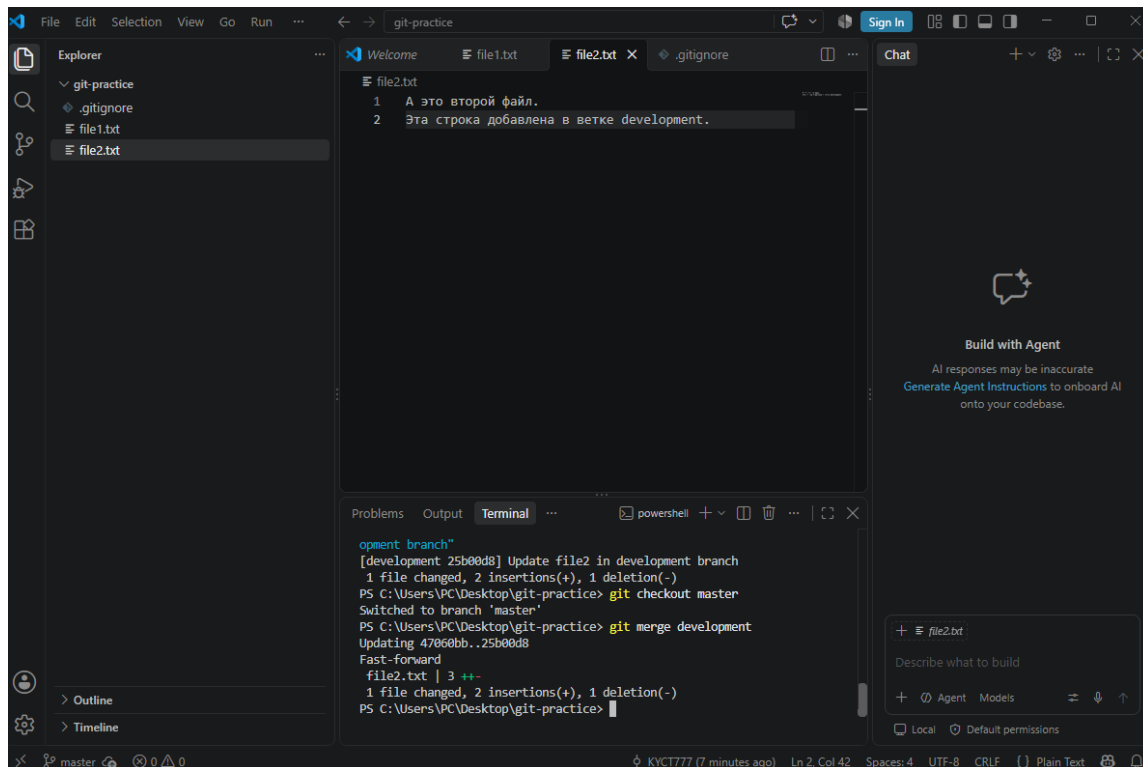


Рисунок 7. Результат пункта 7

Пункт 8. Изменение в ветке разработки и конфликт слияния

Условие

Перейдите обратно в новую ветку и измените тот же файл, что и в основной ветке. Выполните коммит, а затем попробуйте слить ветки. Должен произойти конфликт. Разрешите его вручную и сделайте коммит.

Ход работы

Переключился обратно в ветку development и в том же самом файле file1.txt на той же третьей строчке написал совершенно другой текст. Сделал коммит. Вернулся в master и запустил слияние веток. Git выдал ошибку автоматического объединения и подсветил конфликт. В редакторе VS Code выбрал вариант сохранения изменений из обеих веток (кнопка Accept Both Changes). Маркеры конфликта удалились, строки встали друг за другом. Сохранил файл и закрыл слияние финальным коммитом.

Результат:

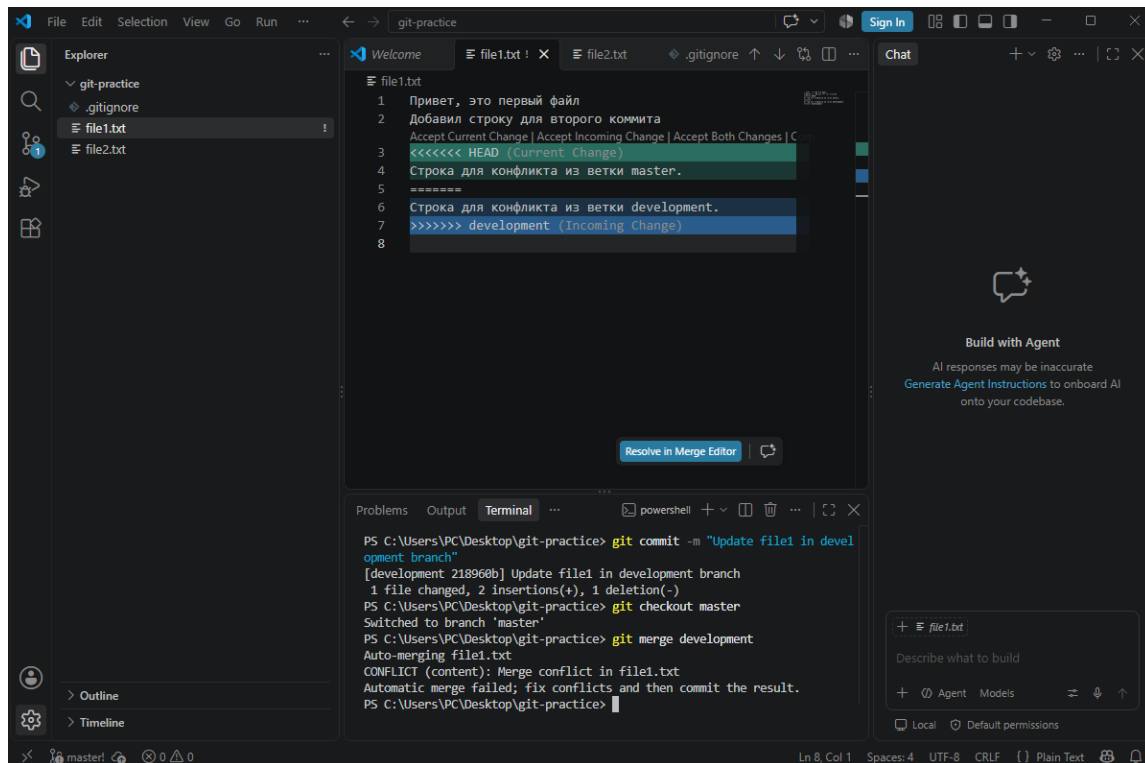


Рисунок 8. Результат пункта 8

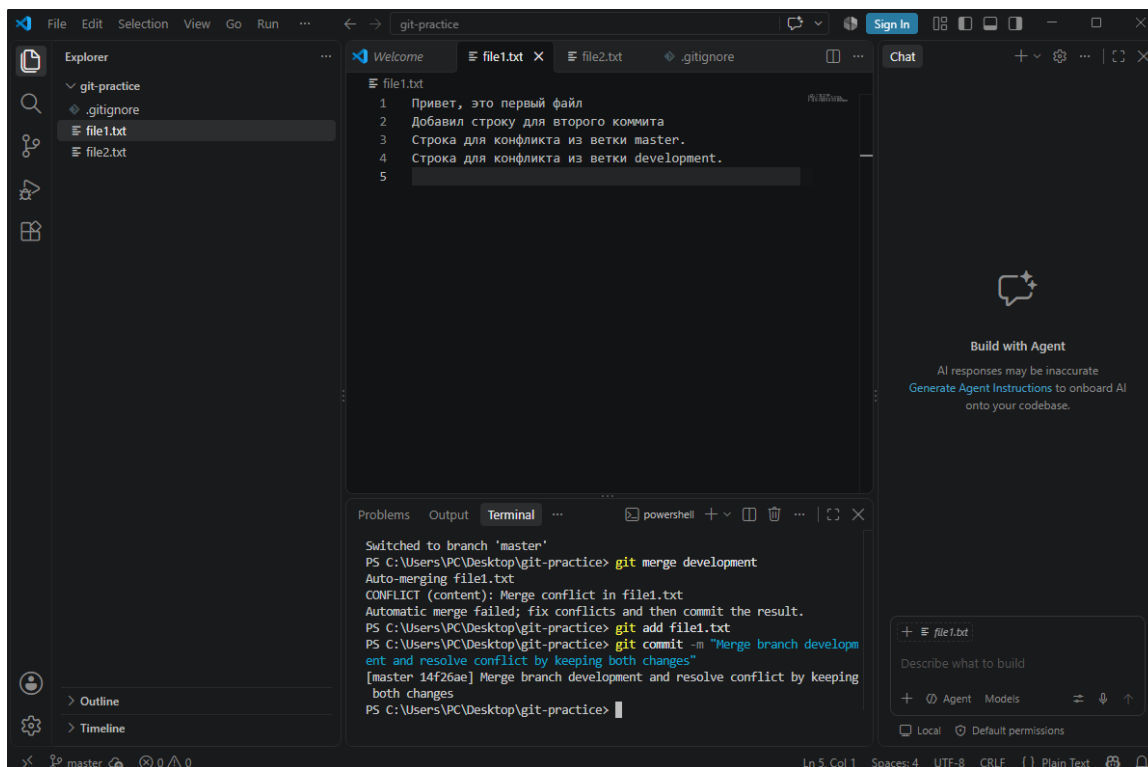


Рисунок 9. Разрешение конфликта в пункте 8

Пункт 9. Просмотр дерева коммитов

Условие

Просмотрите историю коммитов с отображением ветвлений.

Ход работы

Запустил команду просмотра логов с флагами отображения графического дерева. В терминале построился граф, на котором наглядно видно создание ветки, отдельные коммиты в них и последующую точку слияния с конфликтом.

Результат:

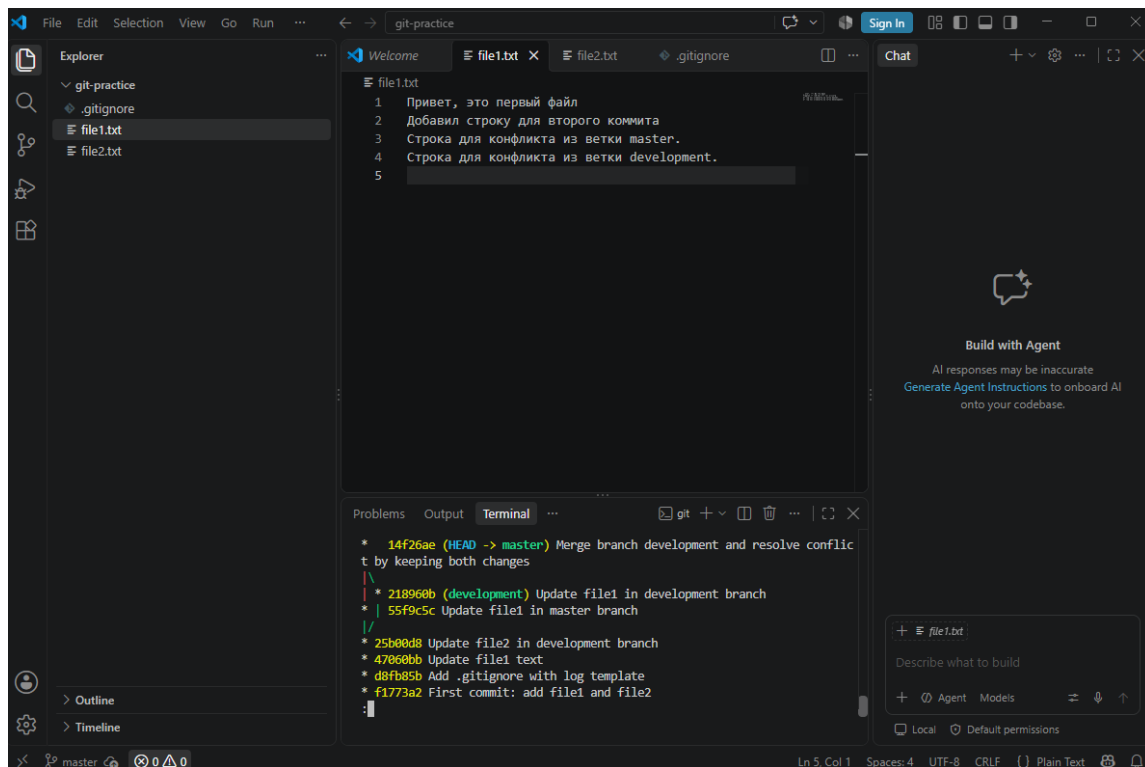


Рисунок 10. Результат пункта 9

Пункт 10. Синхронизация с удаленным репозиторием

Условие

Зарегистрируйтесь на GitHub (если ещё не зарегистрированы) и создайте удалённый репозиторий. Свяжите его с локальным и отправьте в него все коммиты.

Ход работы

Создал новый пустой репозиторий на GitHub. Переименовал локальную ветку в main, связал её с адресом удаленного сервера и отправил всю историю проекта в облако с помощью команды push.

Результат:

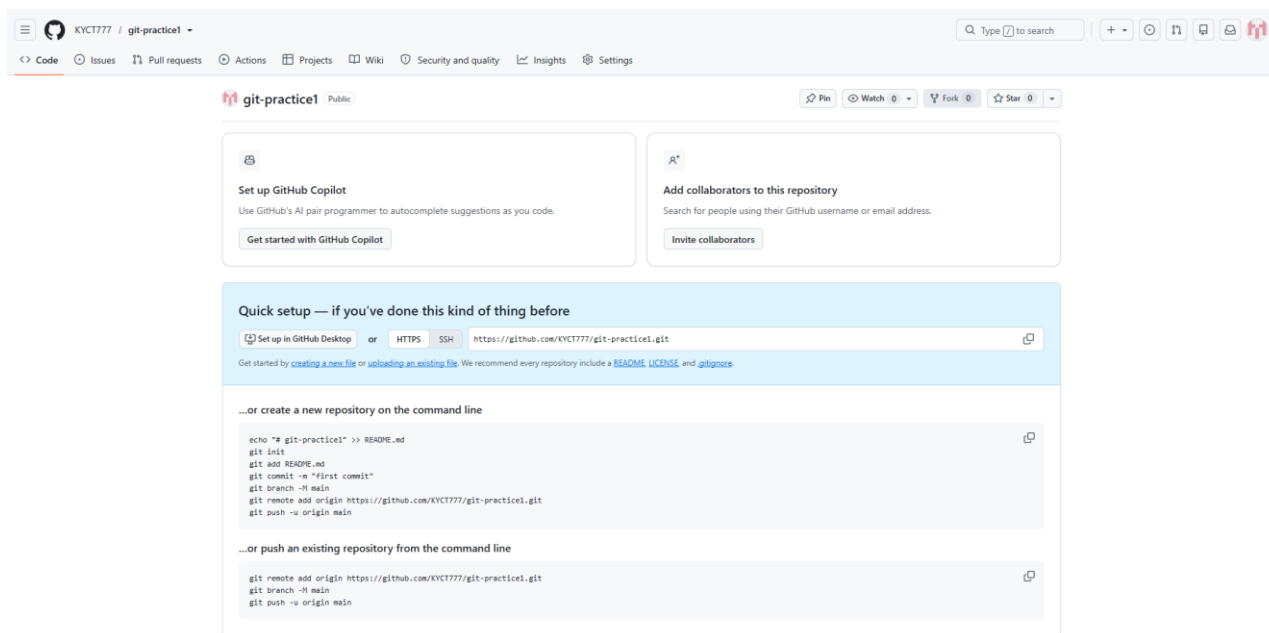


Рисунок 11. Результат пункта 10

Пункт 11. Создание файла README на GitHub

Условие

На GitHub создайте файл README.md с кратким описанием проекта.

Ход работы

Зашел в созданный репозиторий через браузер и воспользовался встроенной кнопкой добавления файла README.md. Написал краткое описание проекта и зафиксировал изменения прямо на сайте..

Результат:

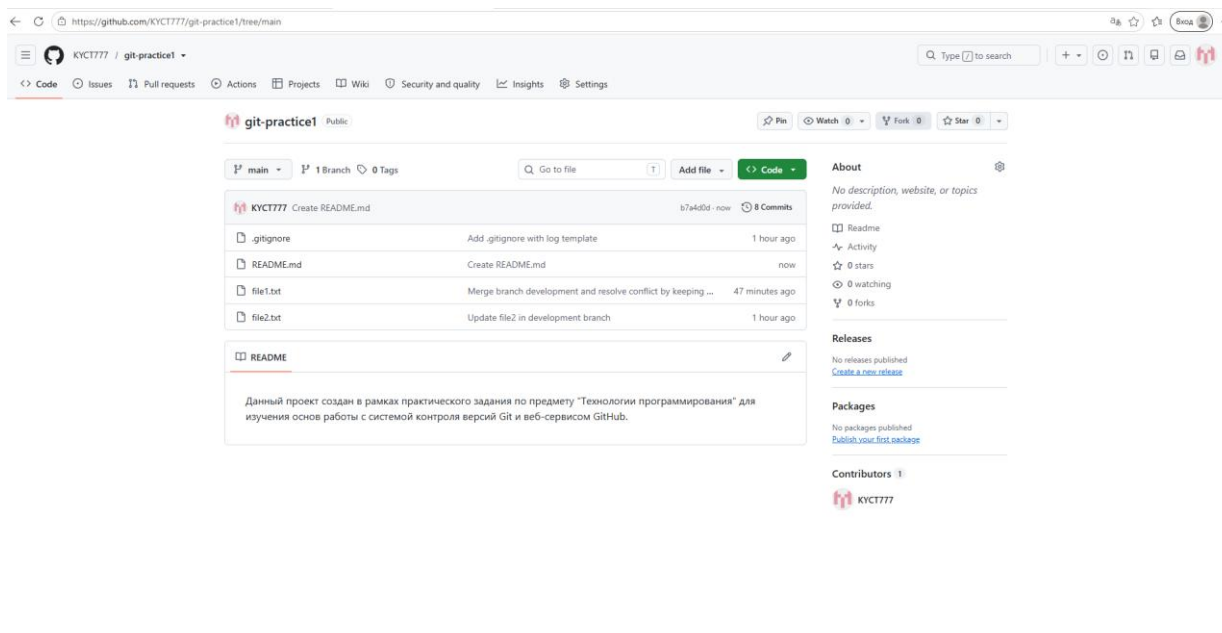


Рисунок 12. Результат пункта 11

Пункт 12. Скачивание изменений на компьютер

Условие

В локальном репозитории получите изменения с GitHub с помощью команды `pull`, чтобы скачать файл `README.md`.

Ход работы

Вернулся в VS Code и выполнил команду `pull`, чтобы забрать добавленный на сайте `README.md` в локальную копию проекта. Файл успешно скачался и появился в структуре папок на компьютере.

Результат:

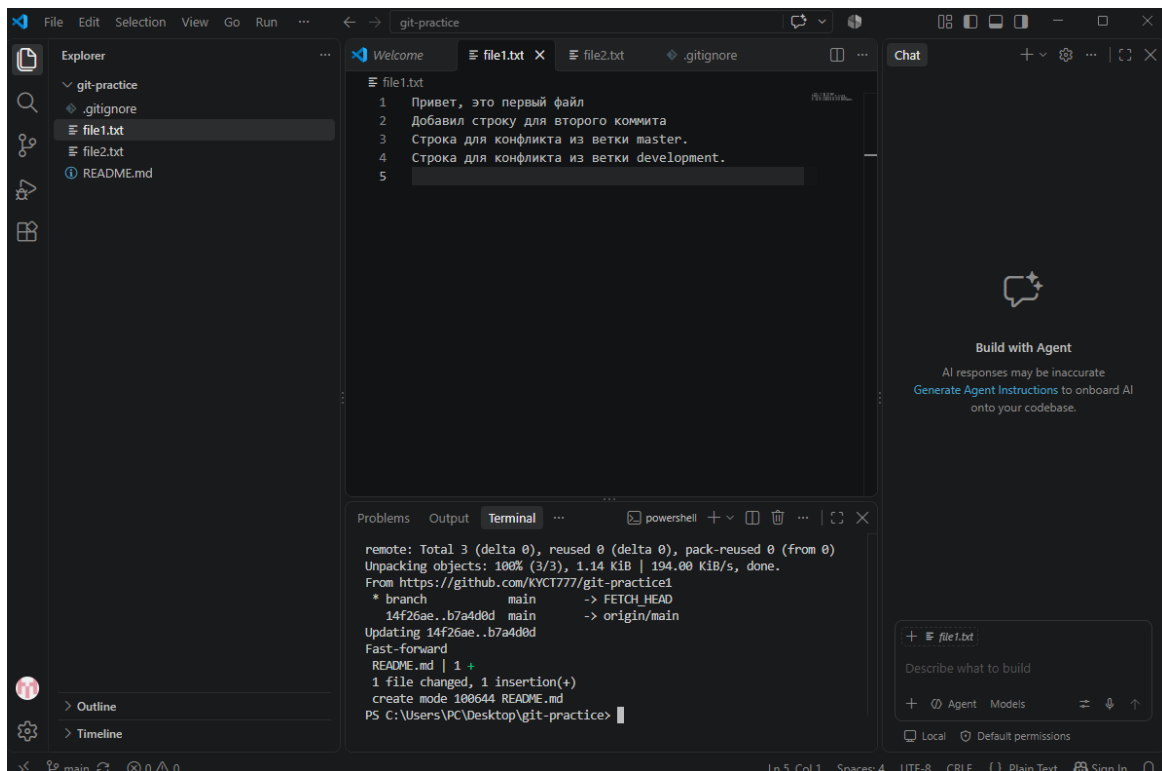


Рисунок 13. Результат пункта 12